

CSIT314 - Weekly Meeting Minutes

Project Title: Event Management System

Prepared by: Chelsea (Project Manager/Documentation Lead)

Meeting Minutes

Date: Sunday, May 25 2025

Time: No formal meeting conducted

Location: Discord (asynchronous updates and check-ins throughout the week)

Attendees:

- Chelsea (Project Manager / Documentation Lead)
- Hafiza (QA Assistant / User Manual Coordinator)
- Sara (Backend Developer / System Integrator)
- Tamim (UML Lead / System Planner)
- Zuhor (Frontend Developer / UI Lead)

Agenda:

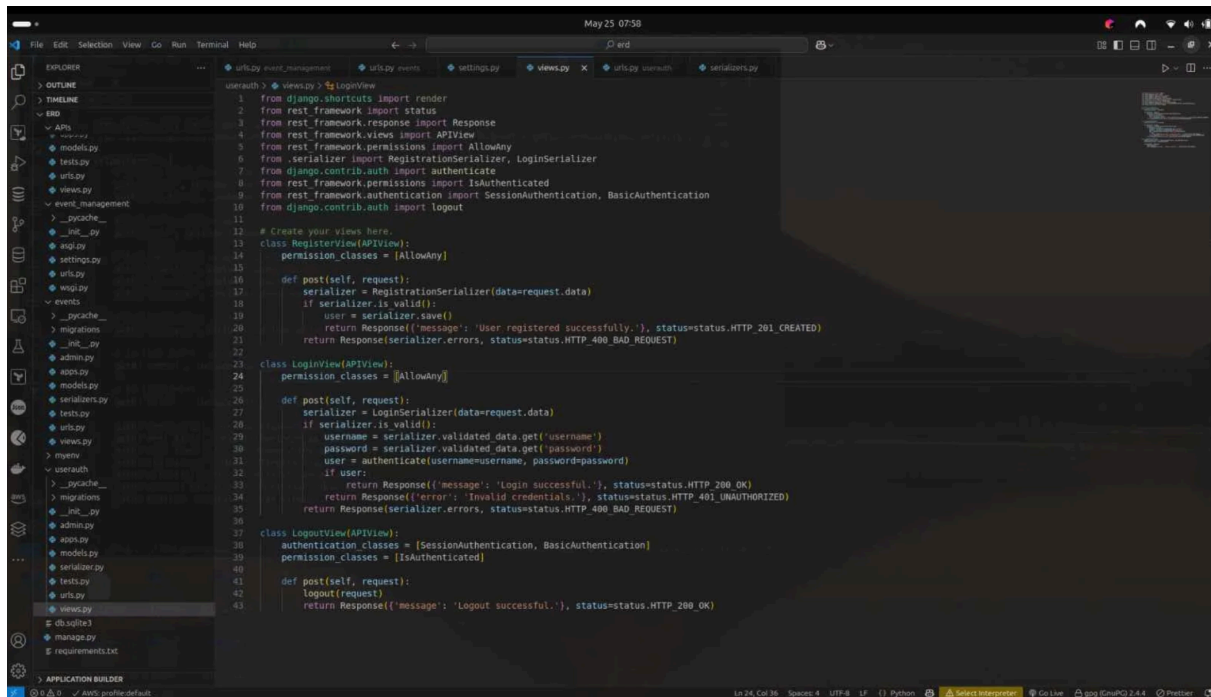
- Review backend organiser code upload
- Reallocate UML + admin planning tasks
- Confirm code implementation status across roles
- Begin feature + reflection documentation
- Finalise VC meeting plan for next week

Progress Summary

Area	Decision / Action	Assigned	Status / Notes
Organiser Backend Review	Code uploaded by Zuhor reviewed; Chelsea provided feedback on logic organisation, duplicate code removal, and route restructuring	Zuhor	In progress; refactor pending
Admin Role Planning	Originally waiting on backend progress for UML updates; re-assigned to discuss admin-side deliverables with Zuhor	Tamim	Reassigned
General User Backend	Shared screenshot of completed code (general user flow); reminder issued to push code to GitHub	Sara	Upload pending, requires review
Organiser Event Creation	Submitted HTML + backend code for organiser event creation; said to be working on user manual, but no evidence provided	Hafiza	Code received, manual unverifiable
Meeting Scheduling	Scheduled VC for following week to discuss test guide allocation and submission prep	Chelsea	Confirmed
Final Documentation Tasks	Progressed work on process log, team roles, challenge notes, and feature coverage in final report	Chelsea	In progress

Notes:

- A real-time VC is scheduled for next week to confirm final deliverable owners and remaining workload coverage.
- Some role tasks (e.g., user manual, admin logic) require confirmation or clearer evidence of progress. GitHub updates from Sara and Hafiza are pending as of Sunday night.
- Backend for organiser logic is being cleaned and modularised after internal code review.
- Documentation writing has begun across multiple final report sections but is not yet complete.



The screenshot shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project structure with folders like 'event_management', 'myenv', 'migrations', 'static', 'templates', and 'users'. The code editor displays the 'views.py' file, which contains three Django REST framework views: 'RegisterView', 'LoginView', and 'LogoutView'. The 'RegisterView' uses the 'RegistrationSerializer' and returns a 201 status for successful registration. The 'LoginView' uses the 'LoginSerializer' and returns a 200 status for successful login. The 'LogoutView' uses 'SessionAuthentication' and 'BasicAuthentication' and returns a 200 status for successful logout. The code is written in Python and uses Django REST framework classes like 'APIView', 'permission_classes', 'serializer', and 'Response'.

```
1 from django.shortcuts import render
2 from rest_framework import status
3 from rest_framework.response import Response
4 from rest_framework.views import APIView
5 from rest_framework.permissions import AllowAny
6 from .serializer import RegistrationSerializer, LoginSerializer
7 from django.contrib.auth import authenticate
8 from rest_framework.permissions import IsAuthenticated
9 from rest_framework.authentication import SessionAuthentication, BasicAuthentication
10 from django.contrib.auth import logout
11
12 # Create your views here.
13 class RegisterView(APIView):
14     permission_classes = [AllowAny]
15
16     def post(self, request):
17         serializer = RegistrationSerializer(data=request.data)
18         if serializer.is_valid():
19             user = serializer.save()
20             return Response({'message': 'User registered successfully.'}, status=status.HTTP_201_CREATED)
21         return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
22
23 class LoginView(APIView):
24     permission_classes = [AllowAny]
25
26     def post(self, request):
27         serializer = LoginSerializer(data=request.data)
28         if serializer.is_valid():
29             username = serializer.validated_data.get('username')
30             password = serializer.validated_data.get('password')
31             user = authenticate(username=username, password=password)
32             if user:
33                 return Response({'message': 'Login successful.'}, status=status.HTTP_200_OK)
34             return Response({'error': 'Invalid credentials.'}, status=status.HTTP_401_UNAUTHORIZED)
35         return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
36
37 class LogoutView(APIView):
38     authentication_classes = [SessionAuthentication, BasicAuthentication]
39     permission_classes = [IsAuthenticated]
40
41     def post(self, request):
42         logout(request)
43         return Response({'message': 'Logout successful.'}, status=status.HTTP_200_OK)
```

Figure 1: Screenshot submitted by Sara showing progress on general user authentication views using Django REST framework. Code includes register, login, and logout functionality. Code is pending push to GitHub and formal review.